ALPE FORMACIÓN · SANTANDER

Edición 2026 · PHP 8.x

CURSO DE PHP · CLASE 34

Autenticación y autorización

- Cómo funciona la autenticación en Laravel
- Los starter kits oficiales
- Nuestro login
- El registro
- Proteger rutas: los middleware auth y guest
- El usuario en las vistas
- Roles
- Gates
- Policies



Ramón Abramo

+25 años de experiencia docente · Santander, Cantabria

ALPE

FORMACIÓN PROFESIONAL

🧭 Contenido de la clase

Al final de la clase 33 os avisé: cualquiera podía crear, editar y borrar cursos. Hoy ponemos dos puertas, y conviene no confundirlas:

- **Autenticación**: ¿**quién** eres? Iniciar sesión con correo y contraseña.
- **Autorización**: ¿**qué** puedes hacer? Un editor puede crear cursos, pero solo el administrador puede borrarlos.

En la clase 13 hicimos un login con sesiones a mano, y en la 26 lo convertimos en la clase `Auth` de nuestro MVC. Laravel trae todo eso hecho y probado, y además un sistema de permisos muy elegante: las *gates* y las *policies*.

•

👌 Cómo vamos a trabajar



1

📘 Cómo funciona la autenticación en Laravel



2

🔍 Los starter kits oficiales



3

👍 Nuestro login



4

🚀 El registro



5

😮 Proteger rutas: los middleware auth y guest



6

👌 El usuario en las vistas



7

🤔 Roles



8

📘 Gates



9

🚀 Policies



10

🚀 Ejercicios



11

👋 Lo que tenéis que llevaros de hoy





CLASE 34

👌 Cómo vamos a trabajar

Copiad el contenido de `ejemplos/` encima del proyecto y ejecutad:

terminal — php-paso-a-paso/clase34 — Visual Studio Code

terminal

```
php artisan migrate:fresh --seed
```

Esquema

```
app/Http/Controllers/Auth/          ← NUEVOS: LoginController y RegistroController
app/Enums/Rol.php                   ← NUEVO: los roles (admin, editor y usuario)
app/Policies/CursoPolicy.php        ← NUEVO: quién puede hacer qué con los cursos
database/migrations/                ← NUEVA: columna rol en users
app/Models/User.php                 ← el rol y los métodos esAdmin() y esEditor()
app/Providers/AppServiceProvider.php ← la gate 'ver-panel'
resources/views/auth/               ← NUEVAS: login y registro
resources/views/admin/panel.blade.php ← NUEVA
resources/views/errors/403.blade.php ← NUEVA: «No tienes permiso»
routes/web.php, CursoController, CursoRequest, el menú y las vistas de cursos
```

El *seeder* crea tres usuarios, uno por rol. La contraseña de los tres es `password`:

Usuario	Correo	Rol
Ramón Abramo	`ramon@academia-alpe.test`	Administrador
Marta Editora	`marta@academia-alpe.test`	Editor
Pablo Usuario	`pablo@academia-alpe.test`	Usuario



💡 Abrid una ventana de incógnito además de la normal: así podéis tener dos usuarios con la sesión abierta a la vez y comprobar qué ve cada uno.



CLASE 34 · PUNTO 1

📘 Cómo funciona la autenticación en Laravel

Por debajo, lo de siempre (clase 13): el usuario envía correo y contraseña, comprobamos la contraseña con `password_verify()` y guardamos su `id` en la **sesión**. En las siguientes peticiones, la cookie de sesión nos dice quién es [1].

En Laravel, todo eso lo hace la fachada `Auth`:

Laravel	Qué hace	Nuestra `Auth` (clase 26)
`Auth::attempt(['email' => ..., 'password' => ...])`	Comprueba los datos e inicia la sesión	`intentar($email, $clave)`
`Auth::login($usuario)`	Inicia la sesión de un usuario concreto	
`Auth::user()` o `auth()->user()`	El usuario identificado (o `null`)	`usuario()`
`Auth::check()`	¿Hay alguien identificado?	`comprobar()`
`Auth::id()`	El `id` del usuario	
`Auth::logout()`	Cierra la sesión	`salir()`

El modelo de los usuarios es `App\Models\User`, que ya existe desde la instalación, y la tabla `users` la creamos con la primera migración. Fijaos en su *cast* `'password' => 'hashed'`: cualquier contraseña que le asignemos se **cifra sola**. Nunca más un `password_hash()` olvidado.



CLASE 34 · PUNTO 2

🔍 Los starter kits oficiales

Laravel ofrece ***starter kits***: proyectos de inicio con login, registro, recuperación de contraseña, verificación del correo y perfil, ya hechos y con un diseño muy cuidado [2]. Se eligen al crear el proyecto con el instalador:

terminal — php-paso-a-paso/clase34 — Visual Studio Code

terminal

```
1 laravel new academia-laravel
```

Starter kit	Frontend
React	React + Inertia + Tailwind
Vue	Vue + Inertia + Tailwind
Livewire	Blade + Livewire + Tailwind

Son estupendos para un proyecto real, pero tienen dos inconvenientes para aprender: necesitan **Node y Vite**, y generan **decenas de ficheros** que todavía no sabríais leer. Por eso hoy hacemos el login **nosotros**, con lo que ya sabemos: rutas, controladores, validación y Blade. Cuando lo entendáis, los *starter kits* os parecerán mucho menos mágicos.



💡 También existe **Laravel Fortify**: toda la lógica de autenticación (incluida la verificación en dos pasos) **sin** vistas. Vosotros ponéis el HTML y él hace el resto.



CLASE 34 · PUNTO 3

👍 Nuestro login

`LoginController.php` tiene tres métodos: mostrar el formulario, procesarlo y cerrar la sesión.

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// POST /login
public function store(Request $request): RedirectResponse
{
    $credenciales = $request->validate([
        'email' => ['required', 'email'],
        'password' => ['required', 'string'],
    ]);

    // attempt() busca el usuario por el email y comprueba la contraseña con
password_verify()
    if (! Auth::attempt($credenciales, $request->boolean('recordarme'))) {
        return back()
            ->withErrors(['email' => 'El correo o la contraseña no son correctos.'])
            ->onlyInput('email');
    }

    // Nuevo identificador de sesión: evita la fijación de sesión (clase 13)
    $request->session()->regenerate();

    // A la página que quería ver antes de que le pidiéramos identificarse, o a la portada
    return redirect()->intended(route('inicio'))
        ->with('exito', '¡Hola, ' . Auth::user()->name . '!');
}
```

- **El mensaje de error no dice qué ha fallado.** «El correo o la contraseña no son correctos» no revela si ese correo existe. Si dijéramos «Ese correo no está registrado», le estaríamos dando pistas a un atacante.
- `withErrors()` crea el error a mano, y `onlyInput('email')` devuelve el correo al formulario, pero **nunca** la contraseña.
- El segundo parámetro de `attempt()` es el **«recordarme»**: guarda una cookie para que la sesión dure semanas (para eso está la columna `remember_token`).
- `redirect()->intended()`: si alguien intentó entrar en `/cursos/create` sin sesión, Laravel lo mandó al login y **apuntó adónde iba**. Después de identificarse, lo devolvemos allí.

Para salir:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// POST /logout
public function destroy(Request $request): RedirectResponse
{
    Auth::logout();

    $request->session()->invalidate();
    $request->session()->regenerateToken();

    return redirect()->route('inicio')->with('exito', 'Has cerrado la sesión.');
}
```



⚠️ El *logout* es un `POST` con `@csrf`, nunca un enlace. Si fuera un `GET`, cualquier web podría cerraros la sesión con una simple imagen: `<img src="https://academia.test/logout">`.



CLASE 34 · PUNTO 4

🚀 El registro

`RegistroController.php`:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// POST /registro
public function store(Request $request): RedirectResponse
{
    $datos = $request->validate([
        'name' => ['required', 'string', 'max:100'],
        'email' => ['required', 'email', 'max:255', 'unique:users,email'],
        'password' => ['required', 'confirmed', Password::min(8)->letters()->numbers()],
    ]);

    // No hace falta password_hash(): el cast 'hashed' del modelo User la cifra al guardar
    $usuario = User::create($datos);

    Auth::login($usuario);
    $request->session()->regenerate();

    return redirect()->route('inicio')->with('exito', "¡Bienvenido, {$usuario->name}!");
}
```

- `confirmed` exige un segundo campo llamado `password_confirmation` con el mismo valor.

- `Password::min(8)->letters()->numbers()` es una regla para contraseñas con requisitos. También tiene `mixedCase()`, `symbols()` y `uncompromised()`, que comprueba si la contraseña ha aparecido en alguna filtración conocida.
- Y aquí está la razón del `#[Fillable]` de la clase 32: el modelo `User` solo deja rellenar `name`, `email` y `password`. Aunque alguien añadiera `rol=admin` al formulario, **se ignoraría**. 😎



Es nuestro middleware `Autenticado` de la clase 26. En Laravel ya viene hecho y se llama `auth` [3]:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// Solo para quien NO ha iniciado sesión
Route::middleware('guest')->group(function () {
    Route::get('/login', [LoginController::class, 'create'])->name('login');
    Route::post('/login', [LoginController::class, 'store'])->middleware('throttle:5,1');
    Route::get('/registro', [RegistroController::class, 'create'])->name('registro');
    Route::post('/registro', [RegistroController::class, 'store']);
});

// Solo para quien SÍ ha iniciado sesión
Route::middleware('auth')->group(function () {
    Route::post('/logout', [LoginController::class, 'destroy'])->name('logout');

    // Crear, editar y borrar cursos (la policy decide además según el rol)
    Route::resource('cursos', CursoController::class)->except(['index', 'show']);

    // El panel, solo para quien pase la gate 'ver-panel'
    Route::view('/admin', 'admin.panel')->middleware('can:ver-panel')->name('admin.panel');
});

// Ver los cursos es público (va después para que /cursos/create no se tome como un {curso})
Route::resource('cursos', CursoController::class)->only(['index', 'show']);
```

Middleware	Si no se cumple…
`auth`	Redirige a la ruta llamada `login` (por eso ese nombre es obligatorio)
`guest`	Si ya estás identificado, te manda a la portada: no tiene sentido ver el login
`throttle:5,1`	Como mucho **5 intentos por minuto**; después, error 429. Frena a quien intente adivinar contraseñas a lo bruto
`can:ver-panel`	Error **403** (lo vemos en el punto 8)



⚠️ **El orden de las rutas importa.** Si la ruta `cursos/{curso}` (la de `show`) estuviera antes que `cursos/create`, Laravel pensaría que «create» es el `{curso}` y buscaría un curso con ese `id`. Por eso registramos primero las rutas protegidas del recurso.

Y hemos quitado la ruta `/usuarios/{user}` de la clase 29: ahora que hay usuarios de verdad, enseñar su correo a cualquiera sería un fallo de privacidad.



CLASE 34 · PUNTO 6

👌 El usuario en las vistas

En Blade tenemos directivas para mostrar cosas según quién mire (`partials/menu.blade.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```
{{-- @auth: solo si hay alguien identificado; @guest: solo si no --}}
@auth
    <span>👤 {{ auth()->user()->name }}</span>
    <form action="{{ route('logout') }}" method="post" style="display: inline">
        @csrf
        <button type="submit" class="boton-secundario">Salir</button>
    </form>
@endauth

@guest
    <a href="{{ route('login') }}">Entrar</a>
    <a href="{{ route('registro') }}">Registro</a>
@endguest
```

Y en los controladores, el usuario está en la petición: `$request->user()`, o en cualquier sitio con `auth()->user()`.



CLASE 34 · PUNTO 7

🤔 Roles

Para saber qué puede hacer cada uno, añadimos a `users` una columna `rol` (migración) y, para no escribir textos sueltos por todo el código, un **enum** como los de la clase 19 (`app/Enums/Rol.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
enum Rol: string
{
    case Admin = 'admin';
    case Editor = 'editor';
    case Usuario = 'usuario';

    public function etiqueta(): string
    {
        return match ($this) {
            self::Admin => 'Administrador',
            self::Editor => 'Editor',
            self::Usuario => 'Usuario',
        };
    }
}
```

En el modelo `User`, un *cast* convierte el texto de la base de datos en el enum, y dos métodos nos hacen la vida fácil:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
protected function casts(): array
{
    return [
        'email_verified_at' => 'datetime',
        'password' => 'hashed',
        'rol' => Rol::class,
    ];
}

public function esAdmin(): bool
{
    return $this->rol === Rol::Admin;
}

public function esEditor(): bool
{
    return $this->rol === Rol::Editor;
}
```



💡 Para proyectos con muchos roles y permisos que cambian desde un panel, existe el paquete **spatie/laravel-permission**, que guarda roles y permisos en tablas propias. Para la academia, una columna y un enum son más que suficientes.



CLASE 34 · PUNTO 8

📘 Gates

Una ***gate*** («puerta») es un **permiso con nombre**: una función que recibe el usuario y devuelve `true` o `false` [4]. Se definen en el `boot()` de `AppServiceProvider`:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// Una gate: un permiso con nombre que no depende de ningún modelo
Gate::define('ver-panel', function (User $user) {
    return $user->esAdmin() || $user->esEditor();
});
```

Y se comprueban así:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// En las rutas: si no la pasa, error 403
Route::view('/admin', 'admin.panel')->middleware('can:ver-panel');

// En el código
if (Gate::allows('ver-panel')) { ... }
Gate::authorize('ver-panel');          // lanza un 403 si no la pasa
$request->user()->can('ver-panel');    // true o false
```

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```blade
{{-- En las vistas --}}
@can('ver-panel')
    <a href="{{ route('admin.panel') }}">Panel</a>
@endcan
```



⚠️ **Ocultar un botón no es proteger.** `@can` solo decide qué se **ve**. Si alguien escribe la dirección a mano, lo que le para es la comprobación en la **ruta** o en el **controlador**. Hay que hacer las dos cosas.



CLASE 34 · PUNTO 9

🚀 Policies

Cuando los permisos giran en torno a un **modelo** (crear cursos, editar **este** curso, borrar **este** curso), se agrupan en una ***policy***: una clase con un método por acción [5].

terminal — php-paso-a-paso/clase34 — Visual Studio Code

terminal

```
php artisan make:policy CursoPolicy --model=Curso
```

`app/Policies/CursoPolicy.php`:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// Quién puede hacer qué con los cursos. Laravel la encuentra sola por el nombre: Curso → CursoPolicy
class CursoPolicy
{
    // Se ejecuta antes que todo lo demás: el administrador puede con todo
    public function before(User $user, string $accion): ?bool
    {
        return $user->esAdmin() ? true : null;
    }

    public function create(User $user): bool
    {
        return $user->esEditor();
    }

    public function update(User $user, Curso $curso): bool
    {
        return $user->esEditor();
    }

    // Borrar, solo el administrador (ya respondió true en before())
    public function delete(User $user, Curso $curso): bool
    {
        return false;
    }
}
```

`before()` es un atajo muy común: si devuelve `true`, ya está permitido; si devuelve `null`, Laravel sigue preguntando al método de la acción.

¿Por qué recibe `update()` el curso si no lo usamos? Porque mañana la regla podría ser «un editor solo puede editar **sus** cursos»: `return $curso->user_id === $user->id;`. Esa es la fuerza de las *policies*.

Se usan igual que las *gates*, indicando el modelo:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// CursoController: si no puede, error 403
Gate::authorize('create', Curso::class);   // crear: no hay curso todavía, se pasa la clase
Gate::authorize('update', $curso);          // editar ESTE curso
Gate::authorize('delete', $curso);
```

En el *Form Request*, el método `authorize()` que dejamos en `true` en la clase 33 por fin tiene trabajo (`CursoRequest.php`):

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```php
// ¿Puede este usuario enviar el formulario? Pregunta a CursoPolicy
public function authorize(): bool
{
    $curso = $this->route('curso');

    return $curso === null
        ? $this->user()->can('create', Curso::class)
        : $this->user()->can('update', $curso);
}
```

Y en las vistas:

ejemplo.php — php-paso-a-paso/clase34 — Visual Studio Code

ejemplo.php

```blade
@can('update', $curso)
    <a class="boton" href="{{ route('cursos.edit', $curso) }}">✏️ Editar</a>
@endcan

@can('create', App\Models\Curso::class)
    <p><a class="boton" href="{{ route('cursos.create') }}">➕ Nuevo curso</a></p>
@endcan
```

Probadlo con los tres usuarios:

	Ver cursos	Crear y editar	Borrar	Panel
Sin sesión	✅	➡️ al login	➡️ al login	➡️ al login
Pablo (usuario)	✅	⛔ 403	⛔ 403	⛔ 403
Marta (editora)	✅	✅	⛔ 403	✅
Ramón (administrador)	✅	✅	✅	✅

Para el error 403 hemos creado nuestra propia página, `errors/403.blade.php`, igual que la del 404 del ejercicio de la clase 30.

	Gate	*Policy*
Para...	Permisos generales	Permisos sobre un modelo
Dónde	`Gate::define()` en `AppServiceProvider`	Una clase en `app/Policies`
Ejemplo	`ver-panel`	`update` de **este** curso
Se comprueba con	`can:ver-panel`, `Gate::authorize('ver-panel')`, `@can('ver-panel')`	`Gate::authorize('update', $curso)`, `@can('update', $curso)`



Las soluciones se copian dentro del proyecto, respetando sus carpetas.

- ☐ **01 · Mi perfil.** Creá la página `/perfil` (solo con sesión), donde cada usuario cambia su nombre, su ciudad y su biografía. Usá la relación `perfil()` de la clase 32 con `updateOrCreate()`. El nombre del usuario en el menú debe llevar a esta página. → `PerfilController.php`, `perfil/edit.blade.php` y `routes/web.php`
- ☐ **02 · Cambiar la contraseña.** En la misma página, un segundo formulario que pida la contraseña actual (regla `current_password`), la nueva y su confirmación. → `PerfilController.php`
- ☐ **03 · Gestión de usuarios.** Creá la *gate* `administrar-usuarios` (solo el administrador) y una página `/admin/usuarios` con la lista de usuarios y un desplegable para cambiar el rol de cada uno (regla `Rule::enum(Rol::class)`). El administrador no puede cambiarse su propio rol. → `UsuarioController.php`, `admin/usuarios.blade.php`, `AppServiceProvider.php`, `menu.blade.php` y `routes/web.php`



🏆 **Reto:** añadí a `cursos` una columna `user_id` con el usuario que lo creó (rellenadla en `store()` con `$request->user()->id`) y cambiad la *policy* para que un editor solo pueda editar **sus** cursos.



CLASE 34 · PUNTO 11

👋 Lo que tenéis que llevaros de hoy

Idea	En una frase
Autenticación	¿Quién eres? Login con sesión
Autorización	¿Qué puedes hacer? *Gates* y *policies*
`Auth::attempt()`	Comprueba correo y contraseña e inicia la sesión
`session()->regenerate()`	Siempre después de iniciar sesión
`redirect()->intended()`	Vuelve a la página que se quería ver
`'password' => 'hashed'`	La contraseña se cifra sola al guardarla
`auth` / `guest` / `throttle`	Solo con sesión / solo sin sesión / límite de intentos
`@auth` , `@guest` , `@can`	Mostrar u ocultar según el usuario (pero proteger en el servidor)
Roles con un enum	`Rol::Admin` , `Rol::Editor` , `Rol::Usuario`
Gate	Permiso con nombre: `Gate::define('ver-panel', ...)`
Policy	Permisos sobre un modelo: `CursoPolicy` con `create` , `update` , `delete`
Starter kits	Login completo ya hecho, con React, Vue o Livewire

📚 Fuentes y documentación oficial

1. Laravel: Authentication
2. Laravel: Starter Kits
3. Laravel: Middleware
4. Laravel: Authorization, Gates
5. Laravel: Authorization, Policies
6. Laravel: Rate Limiting